

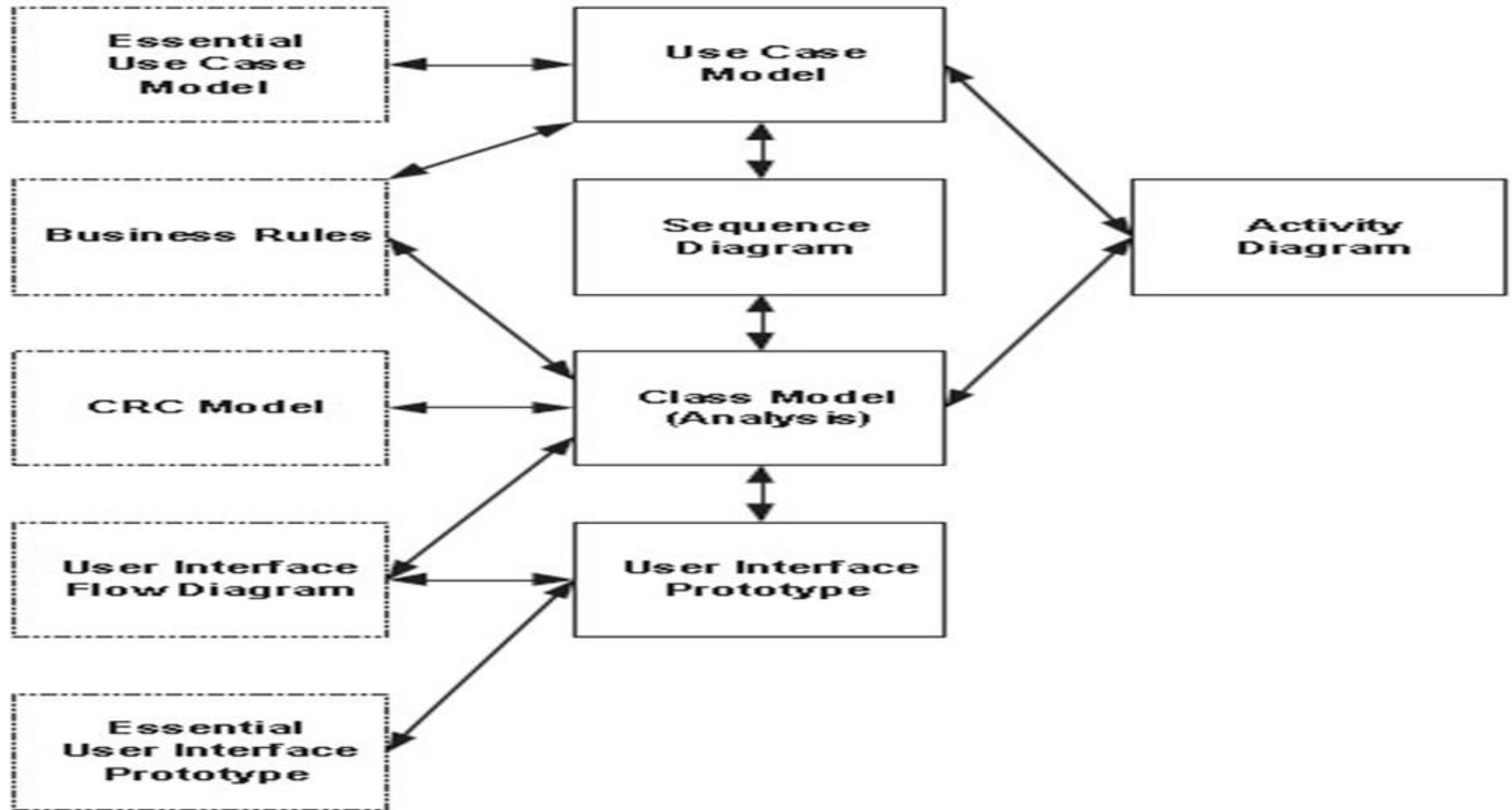
Chapter - FIVE

Determining What to Build: OO Analysis

- ❖ The purpose of the **analysis** is to understand *what will be built*.
- ❖ This is similar to **requirements gathering**.
- ❖ The main difference is that the focus of **requirements gathering is on understanding your users and their potential usage of the system**, whereas the focus of **analysis shifts to understanding the system itself**.

Overview of Analysis artefacts and their Relationships

-
- ❖ The following picture depicts the main artifacts of your analysis efforts and the relationships between them.



Overview of Analysis artefacts and their Relationships...

❖ The picture has three important implications.

1. **Analysis** is an iterative process.

2. *Requirements gathering and analysis are highly interrelated and iterative.*

3. “essential” models, such as the essential **use case model** and **essential user interface prototype**, evolve into corresponding **analysis artifacts** respectively, into the system use case model and user interface prototype.

Overview of Analysis artefacts and their Relationships...

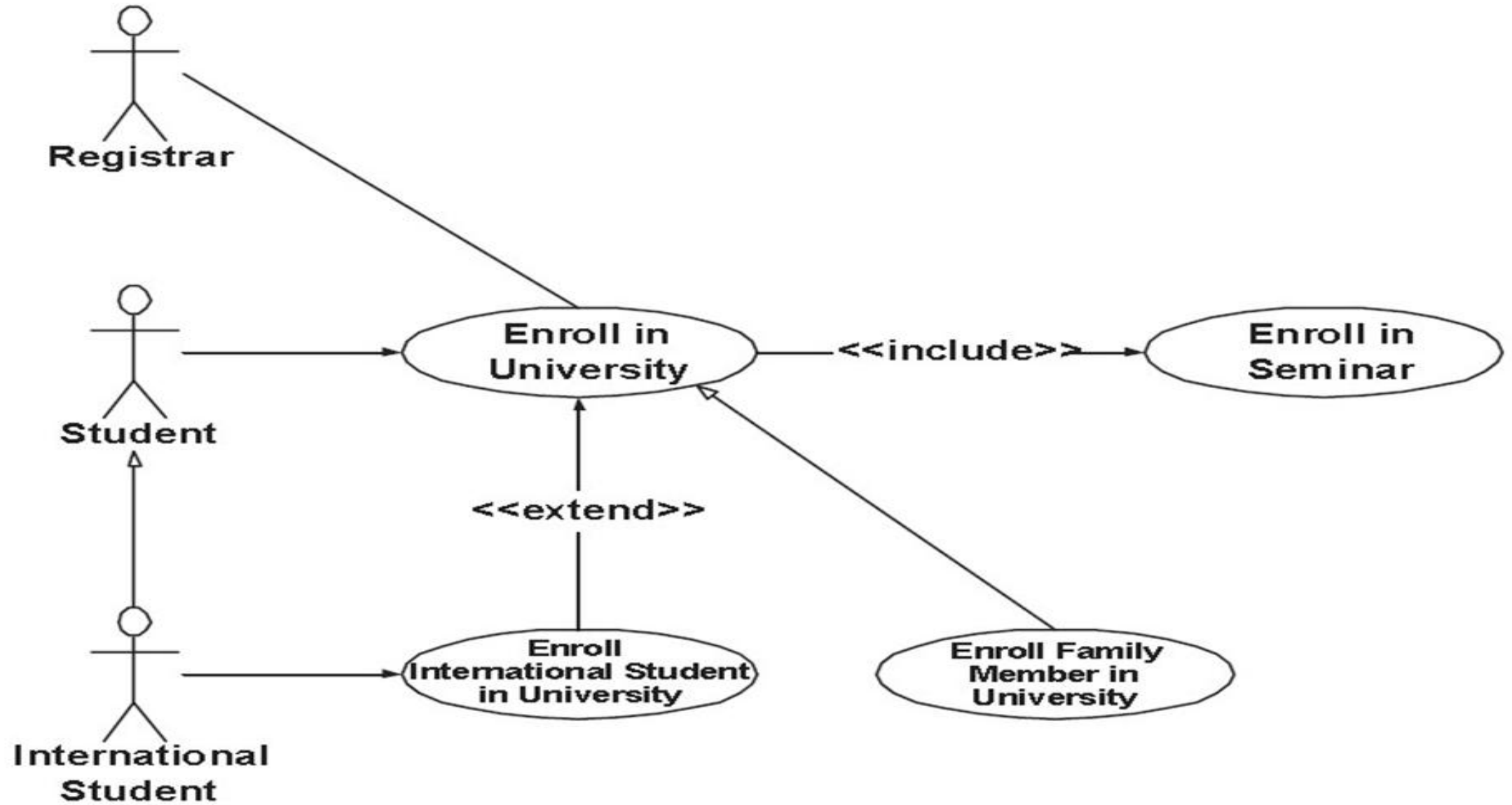
- ❖ During **analysis**, your **main goal** is to evolve your **essential use cases** into **system use cases**.
- ❖ The main difference between an essential use case and a system use case is, in the system use case, you include high-level implementation decisions.
- ❖ – For example, a system use case refers to specific user interface components such as **screens**, **HTML pages**, or **reports**—something you wouldn't do in an essential use case.

Overview of Analysis artefacts and their Relationships...

- ❖ During analysis, you make decisions regarding what will be built or designed.
 - **For example**, a **design decision** is whether your **user interface** is implemented using **browser-based technology**, such as *HTML pages*, or *graphical user interface (GUI) technology such as Windows*.
- ❖ Because your user interface will work differently depending on the implementation technology, the logic of your system use cases, which reflect the flow of your user interface, will also be affected.

Overview of Analysis artefacts and their Relationships...

- ❖ Likewise an essential use case model a system use case model is composed of a use case diagram and the accompanying documentation describing the use cases, actors, and associations.
- ❖ The following(**On Next Slide**) figures provide an example of a use case diagram, depicting a collection of use cases, actors, their associations, a system boundary box (optional), and packages (optional).



Overview of Analysis artefacts and their Relationships...

- ❖ The **rectangle around** the use cases is called the **system boundary** box and, as the name suggests, it delimits the scope of your system.
- ❖ The use cases inside the rectangle represent the functionality you intend to implement.
- ❖ **Finally, packages** are **UML constructs** that enable you to organize model elements (such as use cases) into groups.
- ❖ *Reuse in Use Case Models: <<extend>>, <<include>>, and Inheritance*

Overview of Analysis artefacts and their Relationships...

- ❖ Potential reuse can be modeled through four generalization relationships supported by the UML use case models:
-
- **extend relationships between use cases,**
 - **include relationships between use cases,**
 - **inheritance between use cases,**
 - **inheritance between actors.**

Overview of Analysis artefacts and their Relationships...

- ❖ An important thing to understand about use case models is that **the associations between actors and use cases indicate the need for interfaces.**
- ❖ When an actor is a person, then to support the association, you need to develop user interface components, such as screens and reports.

The Unified Modeling Language (UML)

- ❖ The **Unified Modeling Language (UML)** is a graphical language for OOAD that gives a standard way to write a software system's blueprint.
- ❖ It helps to visualize, specify, construct, and document the artifacts of an object-oriented system.
- ❖ It is used to depict the structures and the relationships in a complex system.

The Unified Modeling Language (UML) ...

➤ Brief History

- ❖ It was developed in the **1990s** as an amalgamation of several techniques, prominently the **OOAD technique** by Grady Booch, OMT (**Object Modeling Technique**) by **James Rumbaugh**, and OOSE (**Object-Oriented Software Engineering**) by Ivar Jacobson.
- ❖ UML attempted to standardize **semantic models**, **syntactic notations**, and **diagrams of OOAD**.

➤ Systems and Models in UML

- ❖ **System:** A set of elements organized to achieve certain objectives form of a system.

The Unified Modeling Language (UML) ...

➤ Systems and Models in UML ...

- ❖ **Model**: A model is a simplified, complete, and consistent abstraction of a system, created for a better understanding of the system.
- ❖ **View**: A view is a projection of a system's model from a specific perspective.

➤ Conceptual Model of UML

- *The Conceptual Model of UML encompasses three major elements:*
 - ✓ **Basic building blocks**
 - ✓ **Rules**
 - ✓ **Common mechanisms**

The Unified Modeling Language (UML) ...

➤ Basic Building Blocks

The three building blocks of UML are:

- ✓ Things
- ✓ Relationships
- ✓ Diagrams

✓ Things

- There are *four kinds of things in UML*, namely:

1. Structural Things: These are the nouns of the UML models representing the static elements that may be either physical or conceptual.

- ✓ The structural things are **class**, **interface**, **collaboration**, use case, active class, components, and nodes.

The Unified Modeling Language (UML) ...

2. Behavioral Things: These are the verbs of the UML models representing the dynamic behavior over time and space. The two types of behavioral things are interaction and state machine.

3. Grouping Things: They comprise the organizational parts of the UML models. There is only one kind of grouping thing, i.e., package.

4. Notational Things: These are the explanations in the UML models representing the comments applied to describe elements.

The Unified Modeling Language (UML) ...

❖ Relationships

- ❖ **Relationships** are the *connection between things*. The four types of relationships that can be represented in UML are:

- ✓ **Dependency**: This is a semantic relationship between two things such that a change in one thing brings a change in the other. The former is the independent thing, while the latter is the dependent thing.
- ✓ **Association**: This is a structural relationship that represents a group of links having a common structure and common behavior.
- ✓ **Generalization**: This represents a **generalization/specialization** relationship in which subclasses inherit structure and behavior from super-classes.
- ✓ **Realization**: This is a *semantic relationship between two or more classifiers* such that one classifier lays down a contract that the other classifiers ensure to abide by.

The Unified Modeling Language (UML) ...

➤ Diagrams

❖ A diagram is a graphical representation of a system. It comprises of a group of elements generally in the form of a graph. *UML includes nine diagrams in all, namely:*

- ✓ **Class Diagram**
- ✓ **Object Diagram**
- ✓ **Use Case Diagram**
- ✓ **Sequence Diagram**
- ✓ **Collaboration Diagram**
- ✓ **State Chart Diagram**
- ✓ **Activity Diagram**
- ✓ **Component Diagram**
- ✓ **Deployment Diagram**

The Unified Modeling Language (UML) ...

➤ Rules

- UML has a number of rules so that the models are semantically self-consistent and related to other models in the system harmoniously. UML has *semantic rules* for the following: **Names**, **Scope**, **Visibility**, **Integrity**, and **Execution**.

➤ Common Mechanisms

- UML has *four common* mechanisms: **Specifications**, **Adornments**, **Common Divisions**, and **Extensibility Mechanisms**.

The Unified Modeling Language (UML) ...

✓ Specifications

- In UML, behind each graphical notation, there is a textual statement denoting the syntax and semantics.
- The specifications provide a semantic backplane that contains all the parts of a system and the relationship among the different paths.

✓ Adornments

- Each element in UML has a unique graphical notation.
- Besides, there are notations to represent the important aspects of an element like name, scope, visibility, etc.

The Unified Modeling Language (UML) ...

❑ Common Divisions

- ✓ Object-oriented systems can be divided in many ways. The **two common** ways of division are:

 - **Division of classes and objects:** A class is an abstraction of a group of similar objects.
- An object is a concrete instance that has actual existence in the system.
 - **Division of Interface and Implementation:** An interface defines the rules for interaction.
- Implementation is the concrete realization of the rules defined in the interface.

The Unified Modeling Language (UML) ...

❑ Extensibility Mechanisms

- UML is an **open-ended language**. It is possible to extend the capabilities of UML in a controlled

manner to suit the requirements of a system. The *extensibility mechanisms* are:

- **Stereotypes:** It extend the vocabulary of the UML, through which new building blocks can be created out of existing ones.
- **Tagged Values:** It extends the properties of UML building blocks.
- **Constraints:** It extends the semantics of UML building blocks.

UML Basic Notations

- UML defines specific notations for each of the building blocks.

➤ **Class**

- A class is represented by a rectangle having **three sections**:
 - ✓ the top section containing the name of the class
 - ✓ the middle section containing class attributes
 - ✓ the bottom section represents operations of the class
- The visibility of the attributes and operations can be represented in the following ways:
- **Public**: A public member is visible from anywhere in the system. In class diagram, it is prefixed by the symbol '+’.

UML Basic Notations

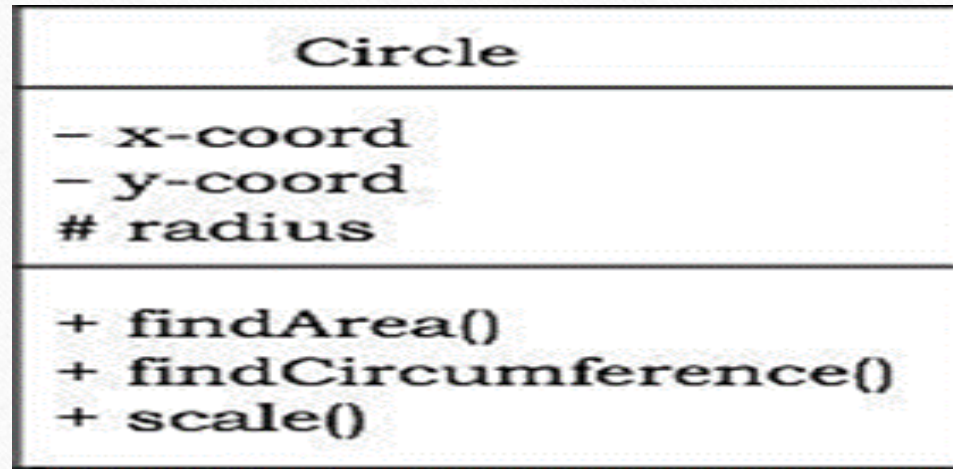
- **Private:** A private member is visible only from within the class. It cannot be accessed from outside the class. A private member is prefixed by the symbol ‘—’.
- **Protected:** A protected member is visible from within the class and from the subclasses inherited from this class, but not from outside. It is prefixed by the symbol ‘#’.

UML Basic Notations

- An abstract class has the class name written in italics.
- Example: Let us consider the Circle class introduced earlier.

- The attributes of Circle are *x-coord*, *y-coord*, and radius.
- The operations are **findArea()**, *findCircumference()*, and **scale()**.
- Let us assume that **x-coord** and **y-coord** are *private data members*, **radius** is a *protected data member*, and *the member functions are public*.
- *The following figure gives the diagrammatic representation of the class.*

UML Basic Notations...



← *Class Name*

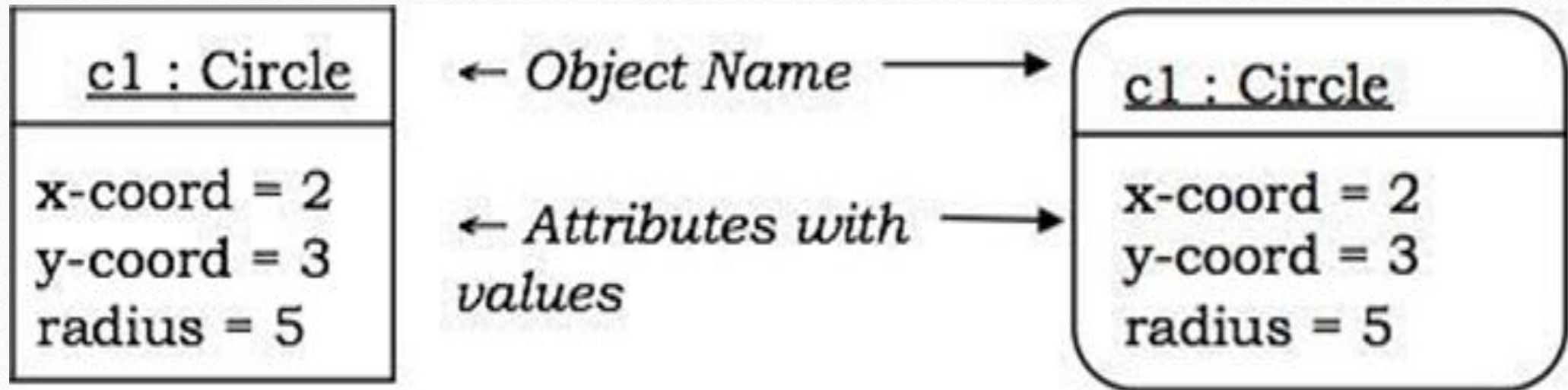
← *Attributes*

← *Operations*

- Object An object is represented as a rectangle with **two** sections:
 - The top section contains the name of the object with the name of the class or package of which it is an instance. The name takes the following forms:
 - ✓ **object-name: class-name**
 - ✓ **object-name : class-name:: package-name**
 - ✓ **class-name: in case of anonymous objects**

UML Basic Notations...

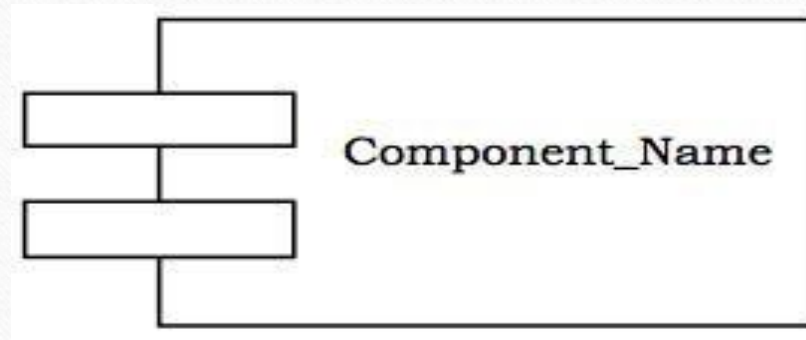
- The bottom section represents the values of the attributes.
- It takes the form **attribute-name = value**. Sometimes objects are represented using rounded rectangles.
- **Example:** Let us consider an object of the class Circle named c1. We assume that the center of c1 is at (2, 3) and the radius of c1 is 5. The following figure depicts the object.



UML Basic Notations...

❑ Component

- A **component** is a physical and replaceable part of the system that conforms to and provides the realization of a set of interfaces.
- It represents the physical packaging of elements like **classes** and **interfaces**.
- ✓ **Notation:** In UML diagrams, *a component is represented by a rectangle* with tabs as shown in the figure below.

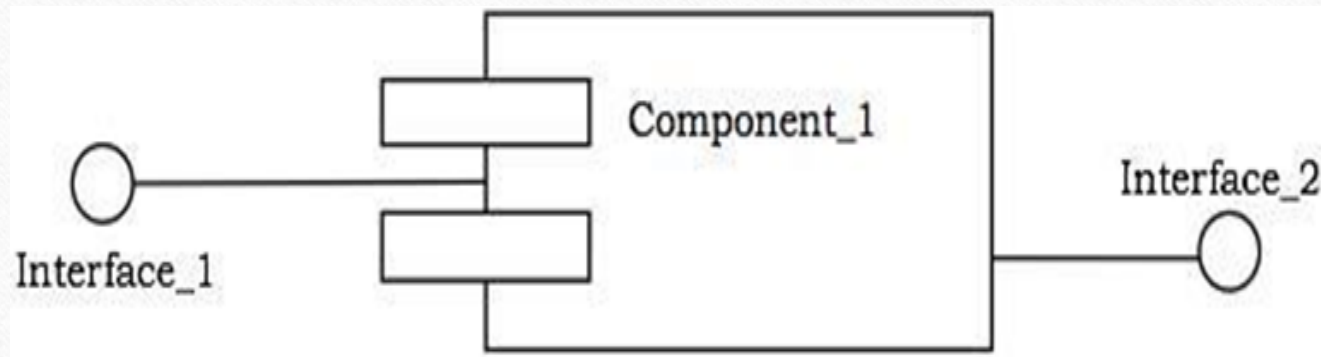


UML Basic Notations...

❑ Interface

- Interface is a collection of methods of a class or component. It specifies the set of services that may be provided by the class or component.

✓ **Notation:** Generally, an **interface** is drawn as a circle together with its name. **An interface** is almost always attached to *the class or component* that realizes it. *The following figure gives the notation of an interface.*

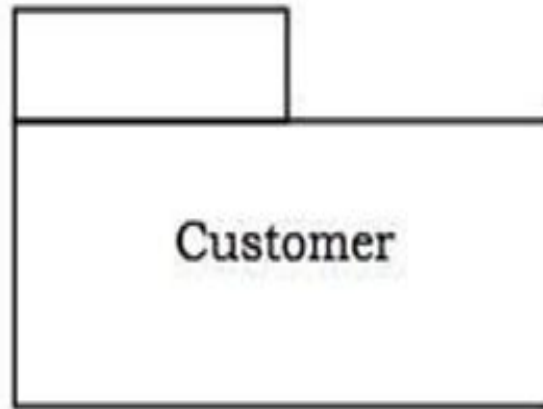


UML Basic Notations...

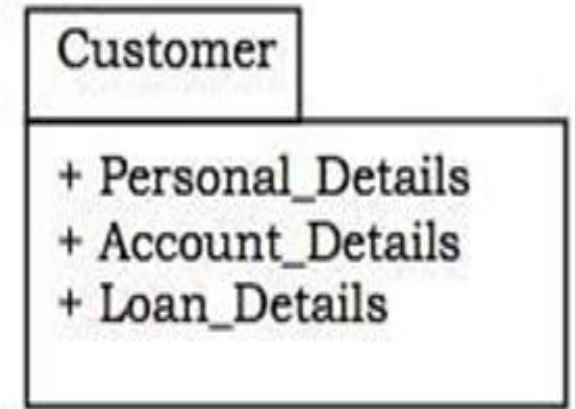
□ Package

- A package is an organized group of elements. A **package** may contain **structural** things like **classes**, **components**, and other **packages in it**.

✓ **Notation:** Graphically, a package is represented by a tabbed folder. A package is generally drawn with only its name. However, it may have additional details about the contents of the package. See the following figures.



(a)

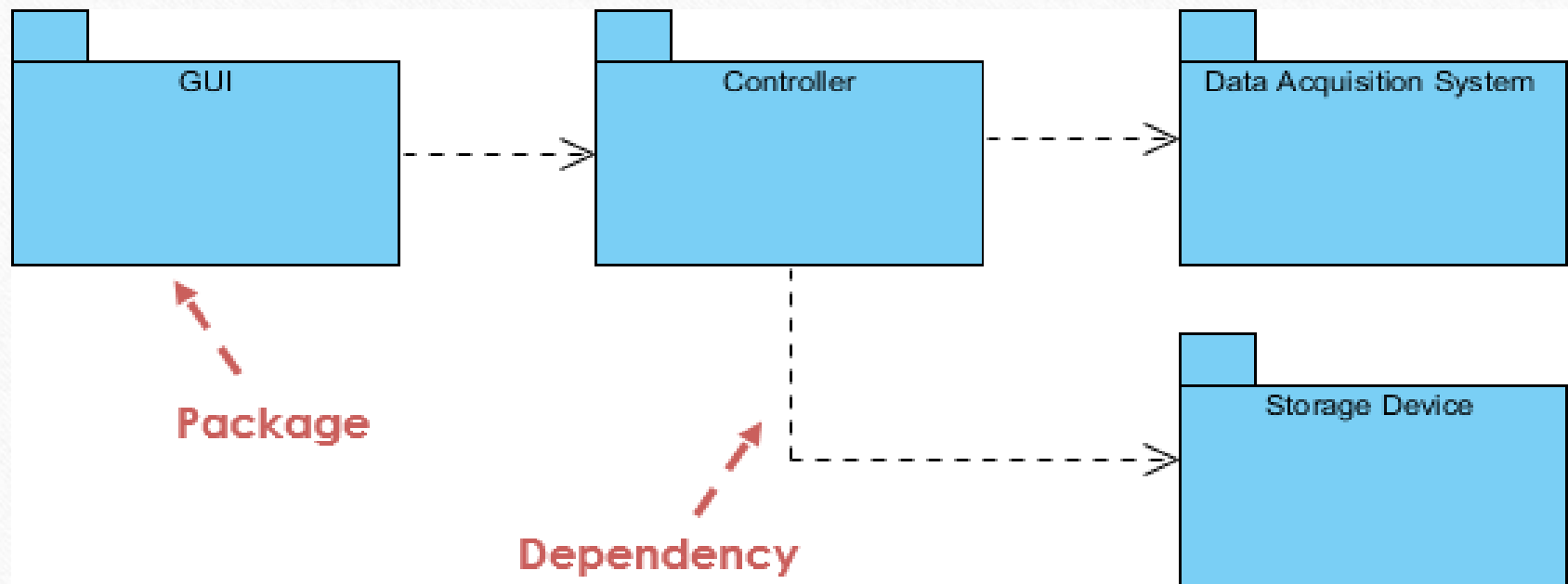


(b)

UML Basic Notations...

□ Package ...

- **Package diagram** is a UML structure diagram that shows packages and dependencies between the packages.
- Model diagrams allow us to show different views of a system, for example, as a multi-layered (aka multi-tiered) application - multi-layered application model.



UML Basic Notations...

Relationship

- The notations for the different types of relationships are as follows:

Dependency



Association



Direct Association



Inheritance



Realization



Aggregation



UML Basic Notations...

- Usually, elements in a relationship play specific roles in the relationship. A role name signifies the behavior of an element participating in a certain context.
-

Example: The following figures show examples of different relationships between classes.

- ✓ The **first figure shows an association between two classes**, **Department** and **Employee**, wherein a department may have a number of employees working in it.
- ✓ The worker is the role name. The ‘1’ alongside **Department** and ‘*’ alongside *Employee* depict that the cardinality ratio is one-to-many.
- ✓ The second figure portrays the **aggregation relationship**, a **University** is the “**whole-of**” many **Departments**.

UML Basic Notations...



(a)



(b)

UML STRUCTURED DIAGRAMS

- UML structural diagrams are categorized as follows: **class diagram**, **object diagram**, **component diagram**, and **deployment diagram**.

✓ **Class Diagram**

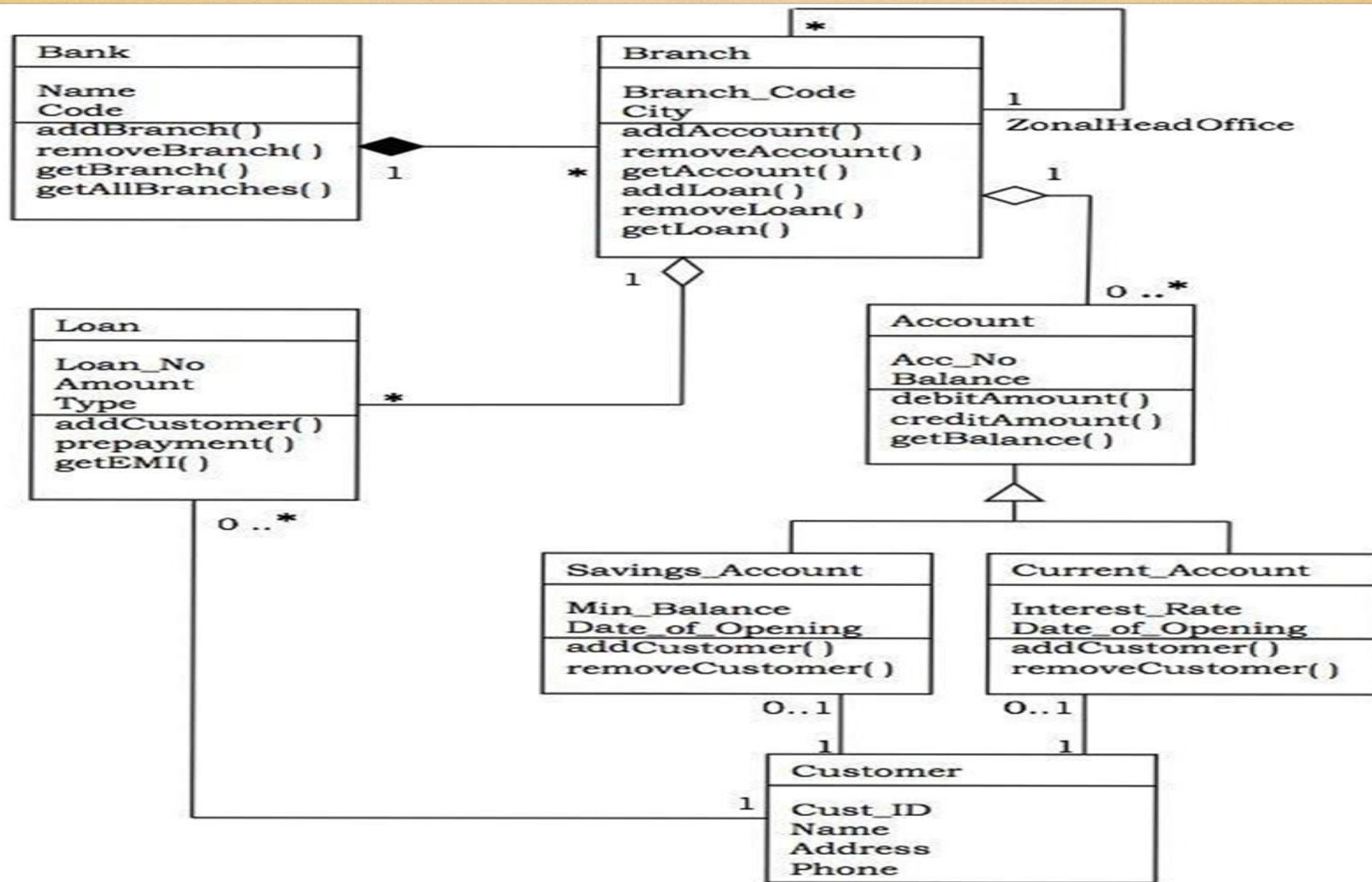
- A class diagram models the static view of a system.
- It comprises of the classes, interfaces, and collaborations of a system; and the relationships between them.

➤ *Class Diagram of a System*

➤ *Let us consider a simplified Banking System.*

UML STRUCTURED DIAGRAMS ...

- A bank has many branches. In each zone, one branch is designated as the zonal head office that supervises the other branches in that zone.
- Each branch can have multiple *accounts and loans*.
- An **account** may be either a savings account or a **current account**.
- A customer may open both a *savings account and a current account*.
- However, a customer must not have more than one savings account or current account.
- A customer may also procure loans from the bank.
 - *The following figure shows the corresponding class diagram.*



UML STRUCTURED DIAGRAMS ...

❖ *Classes in the System*

➤ *Bank, Branch, Account, Savings Account, Current Account, Loan, and Customer.*

❖ *Relationships*

- A Bank “has-a” number of Branches: **composition**, **one-to-many**
 - A Branch with the role of Zonal Head Office supervises other Branches: **unary association**, **one-to-many**.
 - A Branch “has-a” number of accounts: **aggregation**, **one-to-many**.
- ❖ From the **class Account**, **two** classes have *inherited*, namely, **Savings Account** and **Current Account**.
- A Customer can have one Current Account: **association**, **one-to-one**.

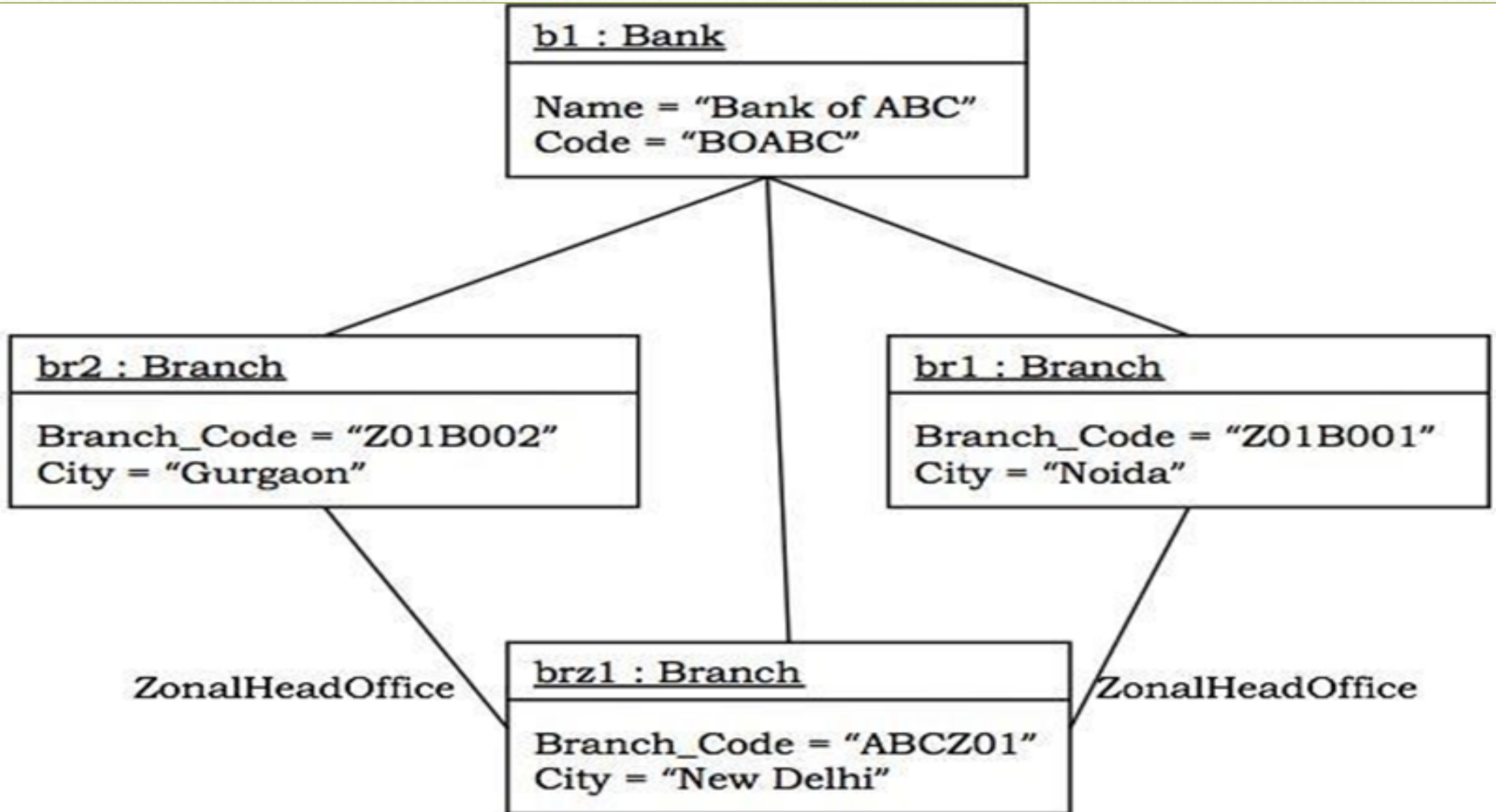
UML STRUCTURED DIAGRAMS ...

- A Customer can have one **Savings Account**: **association**, one-to-one.
 - A Branch “has-a” number of Loans: **aggregation**, one-to-many.
-
- A Customer can take many loans: **association**, one-to-many.

UML STRUCTURED DIAGRAMS ...

✓ **Object Diagram**

- ✓ An object diagram models a group of objects and their links at a point in time.
- ✓ It shows the instances of the things in a class diagram. An object diagram is the static part of an interaction diagram.
- ✓ *Example: The following figure shows an object diagram of a portion of the class diagram of the Banking System.*



UML STRUCTURED DIAGRAMS ...

■ **Component Diagram**

- **Component diagrams** show the **organization** and **dependencies** among a group of components. Component diagrams comprise of:

- **Components**
- **Interfaces**
- **Relationships**
- **Packages and Subsystems (optional)** Component diagrams are used for:
- **Constructing systems through forward and reverse engineering.**
- **Modeling configuration management of source code files while developing a system using an object-oriented programming language.**
- **Representing schemas in modeling databases.**
- **Modeling behaviors of dynamic systems.**

UML STRUCTURED DIAGRAMS ...

■ Deployment Diagram

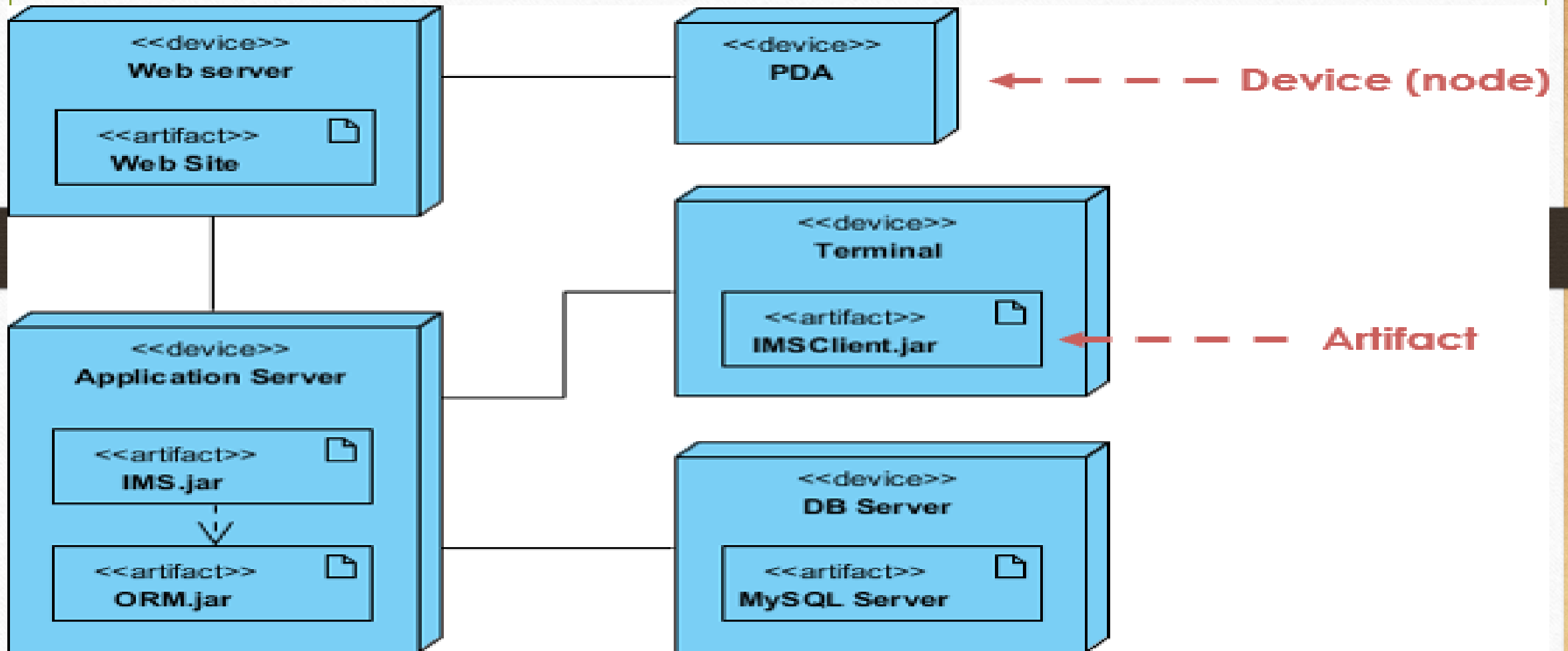
- A **deployment diagram** puts emphasis on the *configuration of runtime processing nodes* and *their components that live on them*.
- They are commonly comprised of nodes and dependencies, or associations between the nodes.
- *Deployment diagrams are used to:*
 - model devices in embedded systems that typically comprise of software-intensive collection of hardware.
 - represent the topologies of client/server systems.
 - model fully distributed systems.

UML STRUCTURED DIAGRAMS ...

- ✓ The **Deployment Diagram** helps **to model the physical aspect of an Object-Oriented software system.**
- ✓ It is a **structure diagram** which shows *architecture of the system as deployment (distribution) of software artifacts to deployment targets.*
- ✓ **Artifacts** represent concrete elements in the physical world that are the result of a development process.
- ✓ It models the run-time configuration in a static view and visualizes the distribution of artifacts in an application.
- ✓ In most cases, it involves modeling the *hardware configurations* together with the *software components that lived on.*

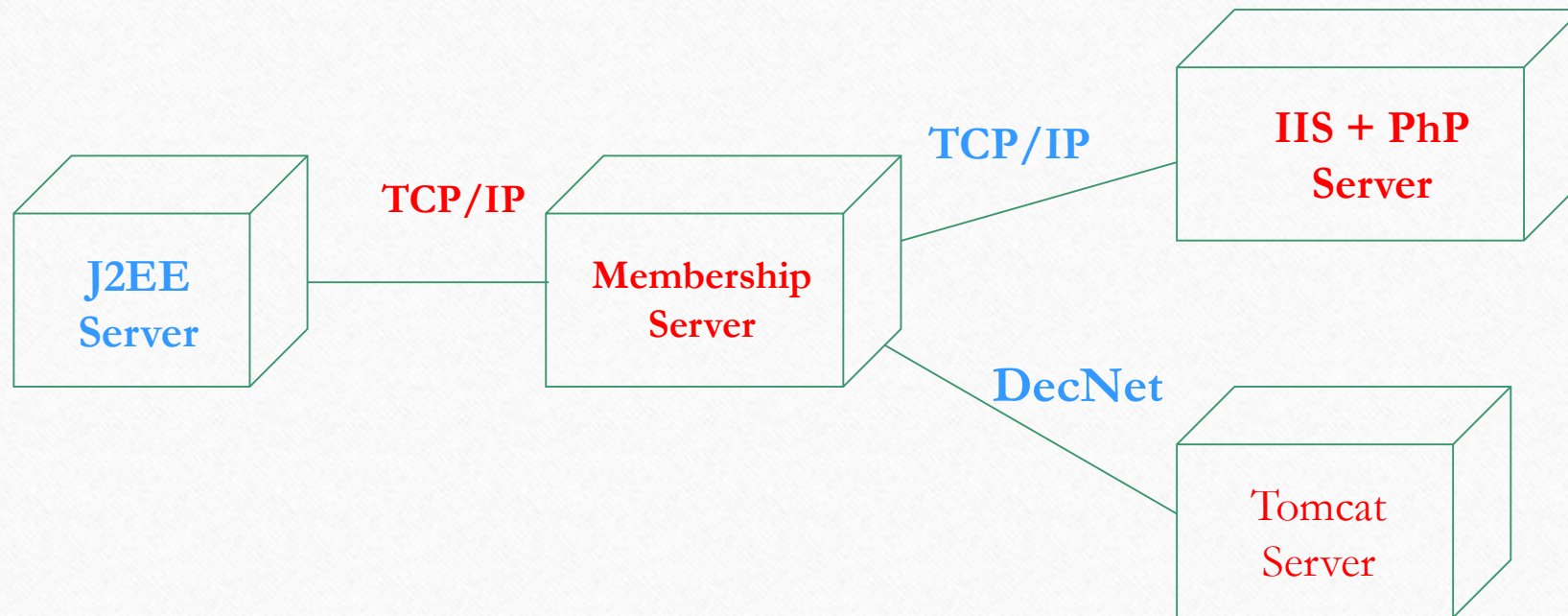
UML STRUCTURED DIAGRAMS ...

■ *Deployment Diagram Example - 1*

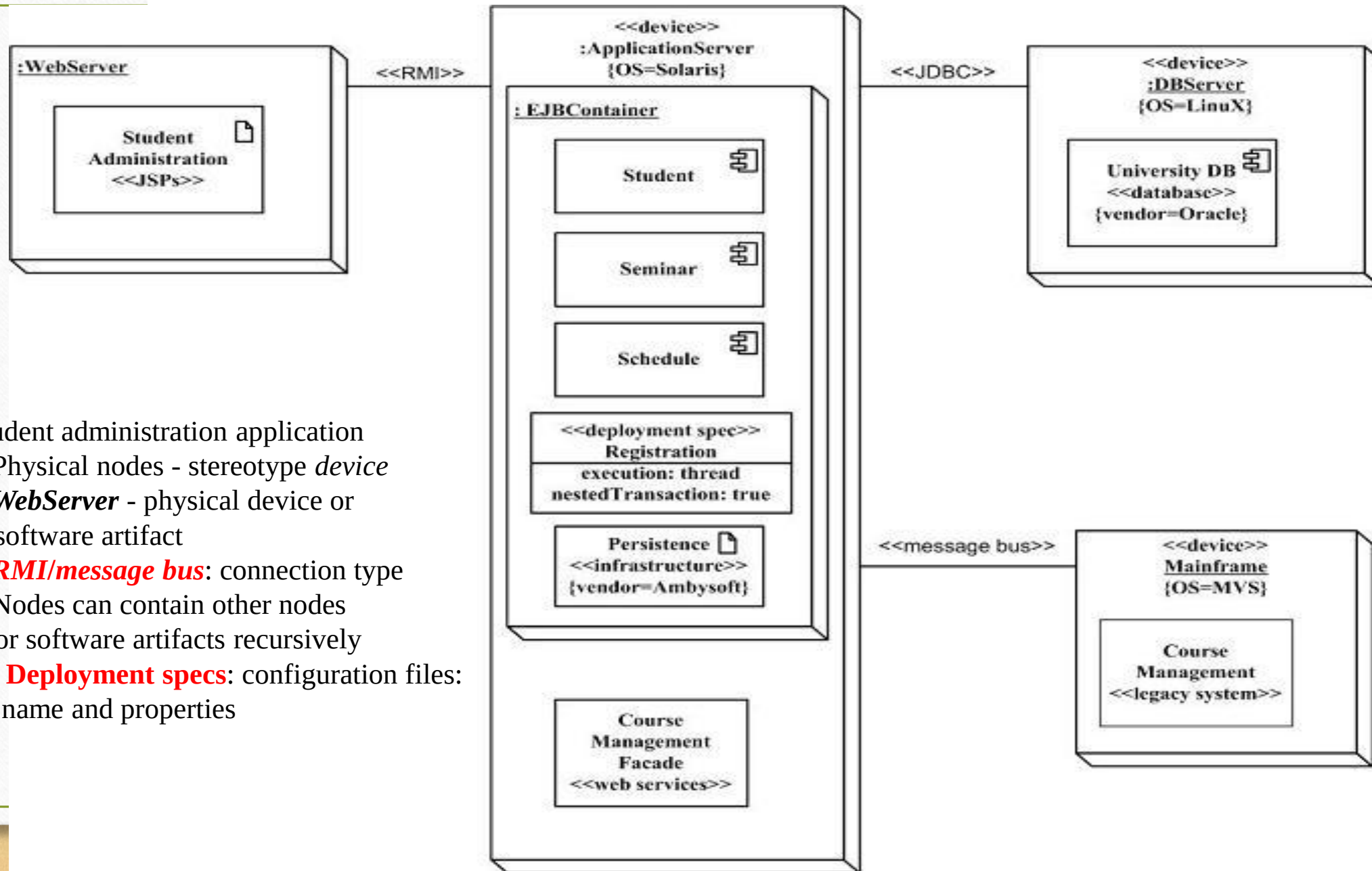


Deployment Diagram....

- Shows a set of ***processing*** nodes and their relationships.
 - Represents the static deployment view of an ***architecture***.
 - Nodes typically enclose one or more components.
-



Deployment Diagram...



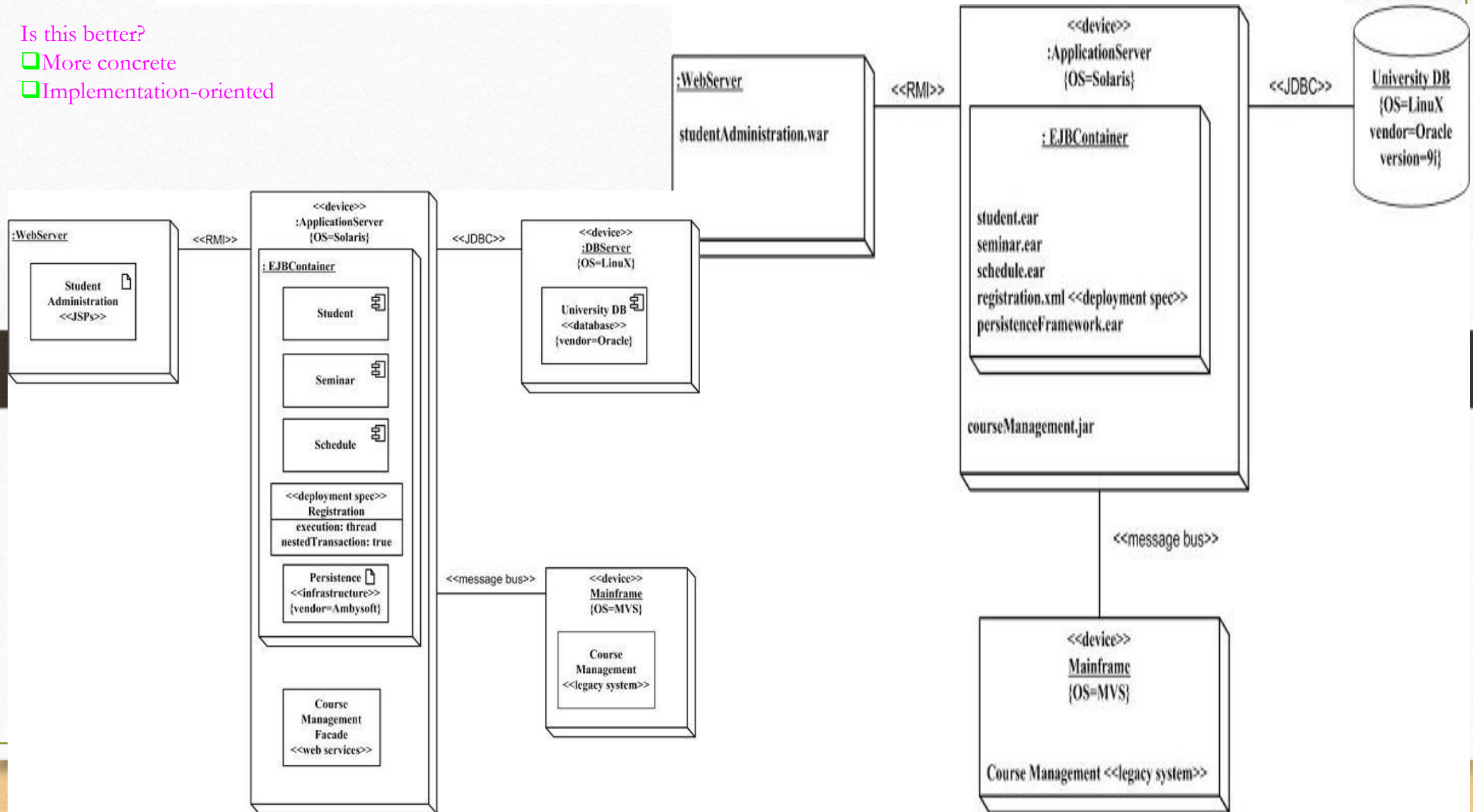
Student administration application

- Physical nodes - stereotype *device*
- WebServer** - physical device or software artifact
- RMI/message bus**: connection type
- Nodes can contain other nodes or software artifacts recursively
- Deployment specs**: configuration files: name and properties

Deployment Diagram...

Is this better?

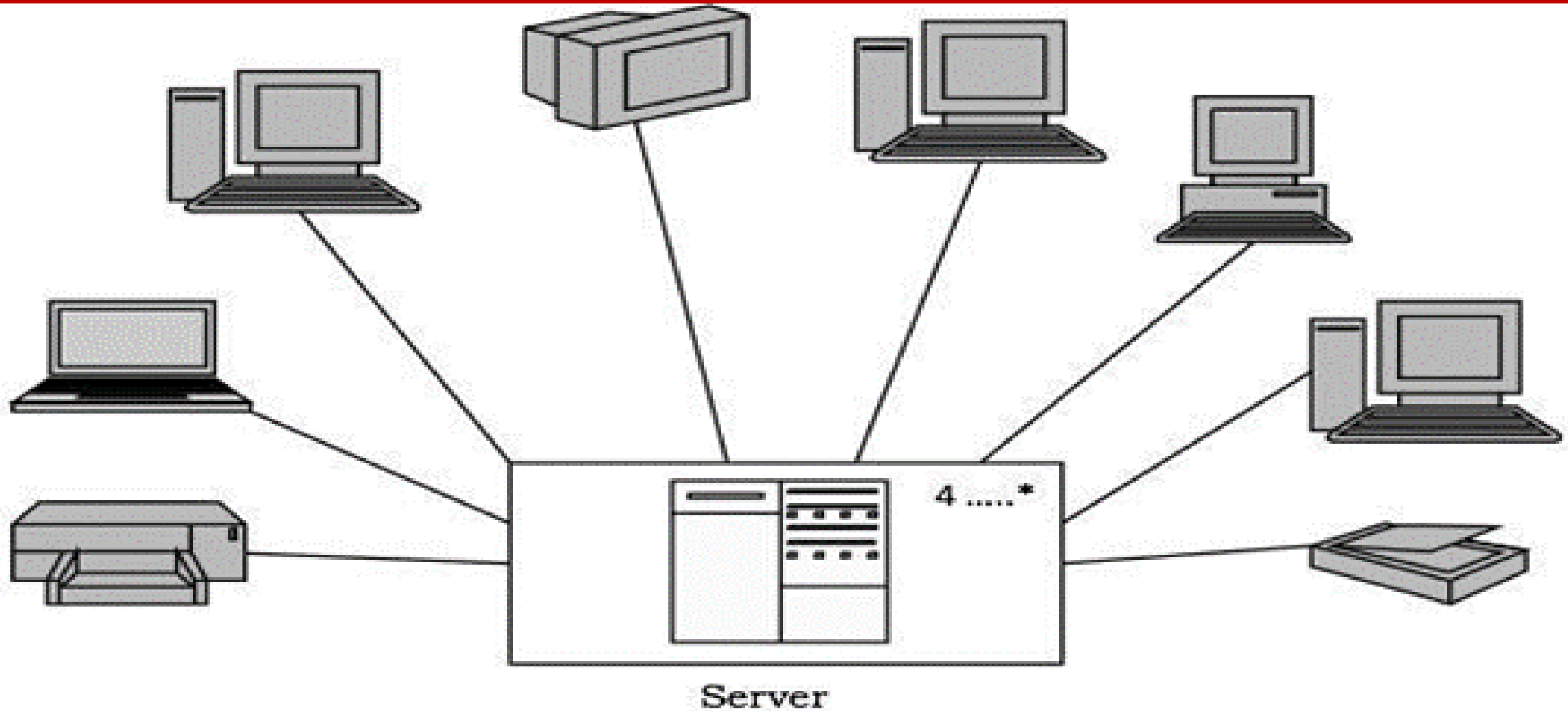
- More concrete
- Implementation-oriented



UML STRUCTURED DIAGRAMS ...

- *Deployment Diagram Example – 2*
- The following figure shows the topology of a computer system that follows **client/server architecture**.
- The figure illustrates a node stereotyped as server that comprises of processors.
- The figure indicates that four or more servers are deployed at the system.
- Connected to the server are the client nodes, where each node represents a terminal device such as **workstation**, **laptop**, **scanner**, or **printer**.
- The nodes are represented using icons that clearly depict the real-world equivalent.

UML STRUCTURED DIAGRAMS ...

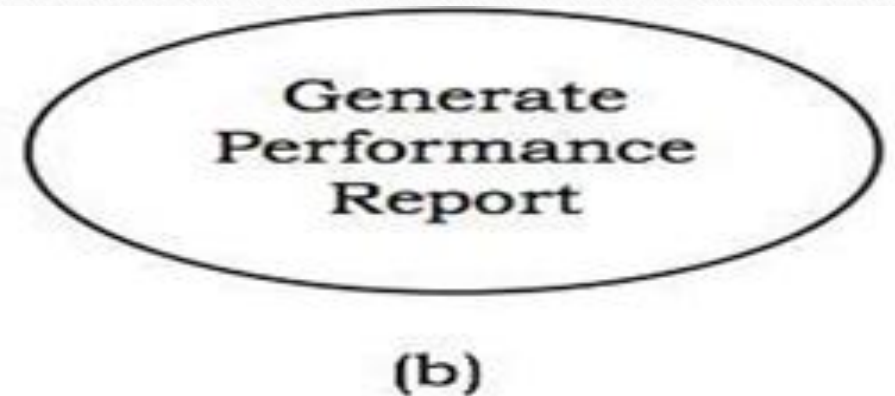
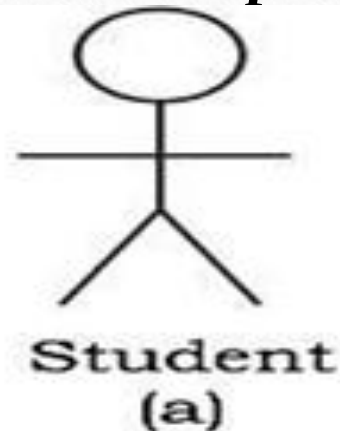


UML Behavioral Diagrams

- **UML behavioral diagrams** visualize, **specify**, **construct**, and **document** the dynamic aspects of a system.
- The behavioral diagrams are categorized as follows: **use case diagrams**, interaction diagrams, **state-chart diagrams**, and **activity diagrams**.
- **Use Case Model**
- **Use Case**
- A **use case** describes the sequence of actions a system performs yielding visible results.
- It shows the interaction of things outside the system with the system itself.
- Use cases may be applied to the whole system as well as a part of the system.

UML Behavioral Diagrams...

- **Actor**
- An **actor** represents the roles that the users of the use cases play.
- An actor may be a person (e.g. student, customer), a device (e.g. workstation), or another system (e.g. bank, institution).
- *The following figure shows the notations of an actor named Student and a use case called Generate Performance Report.*



UML Behavioral Diagrams...

■ Use Case Diagrams

- Use case diagrams present an outside view of the manner the elements in a system behave and how they can be used in the context.
- Use case diagrams comprised of:
 - **Use cases**
 - **Actors**
 - **Relationships like dependency, generalization, and association Use case**
- diagrams are used: to model the context of a system by enclosing all the activities of a system within a rectangle and focusing on the actors outside the system by interacting with it.
- To model the requirements of a system from the outside point of view.

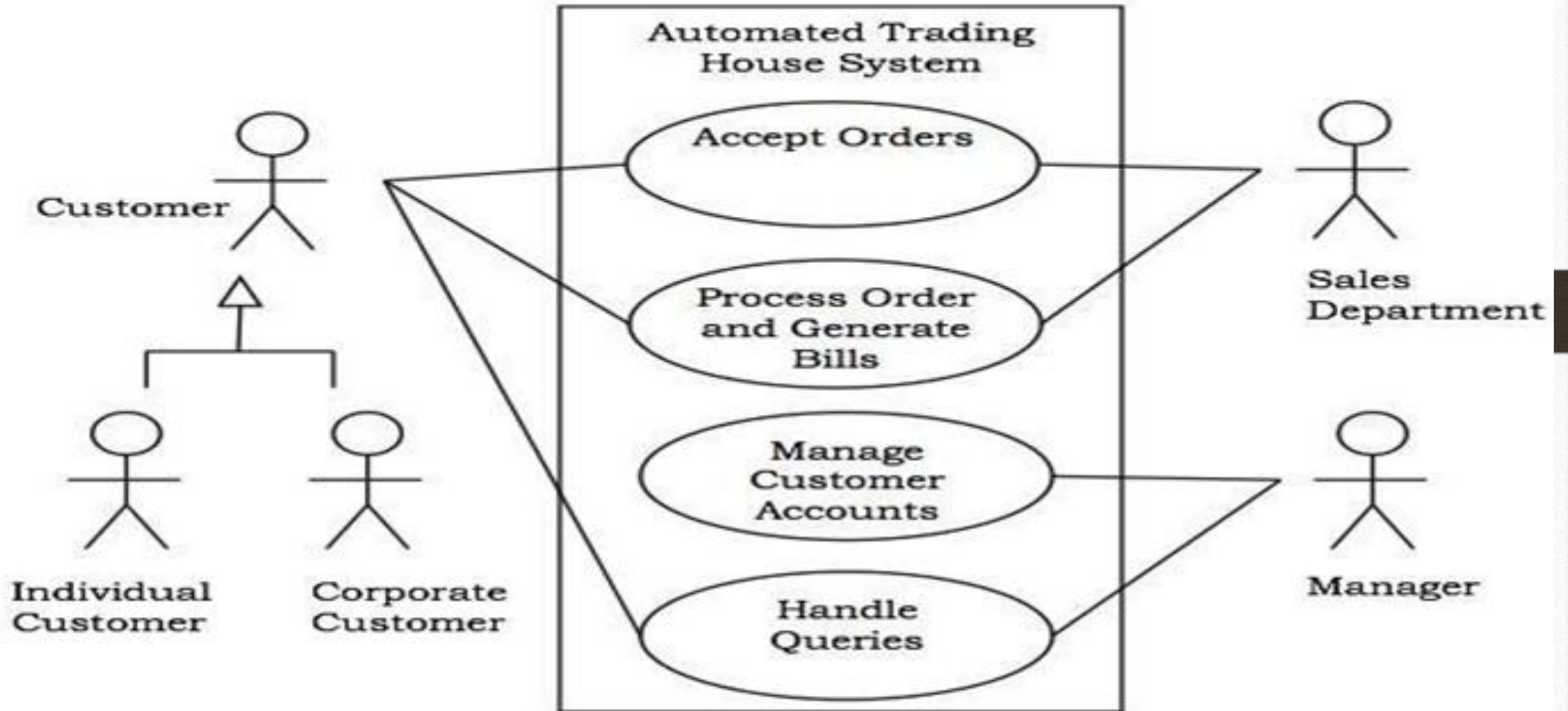
UML Behavioral Diagrams...

✓ *Example*

Let us consider an **Automated Trading House System**. We assume the following features of the system:

- The trading house has transactions with two types of customers, individual customers and corporate customers.
- Once the customer places an order, it is processed by the sales department and the customer is given the bill.
- The system allows the manager to manage customer accounts and answer any queries posted by the customer.

UML Behavioral Diagrams...



UML Behavioral Diagrams...

■ Interaction Diagrams

- **Interaction diagrams** depict interactions of **objects** and **their relationships**. They also include the *messages passed between them*. There are **two** types of interaction diagrams:
 - **Sequence Diagrams**
 - **Collaboration Diagrams**
- **Interaction diagrams are used for modeling:**
 - ✓ the control flow by time ordering using sequence diagrams.
 - ✓ the control flow of organization using collaboration diagrams.

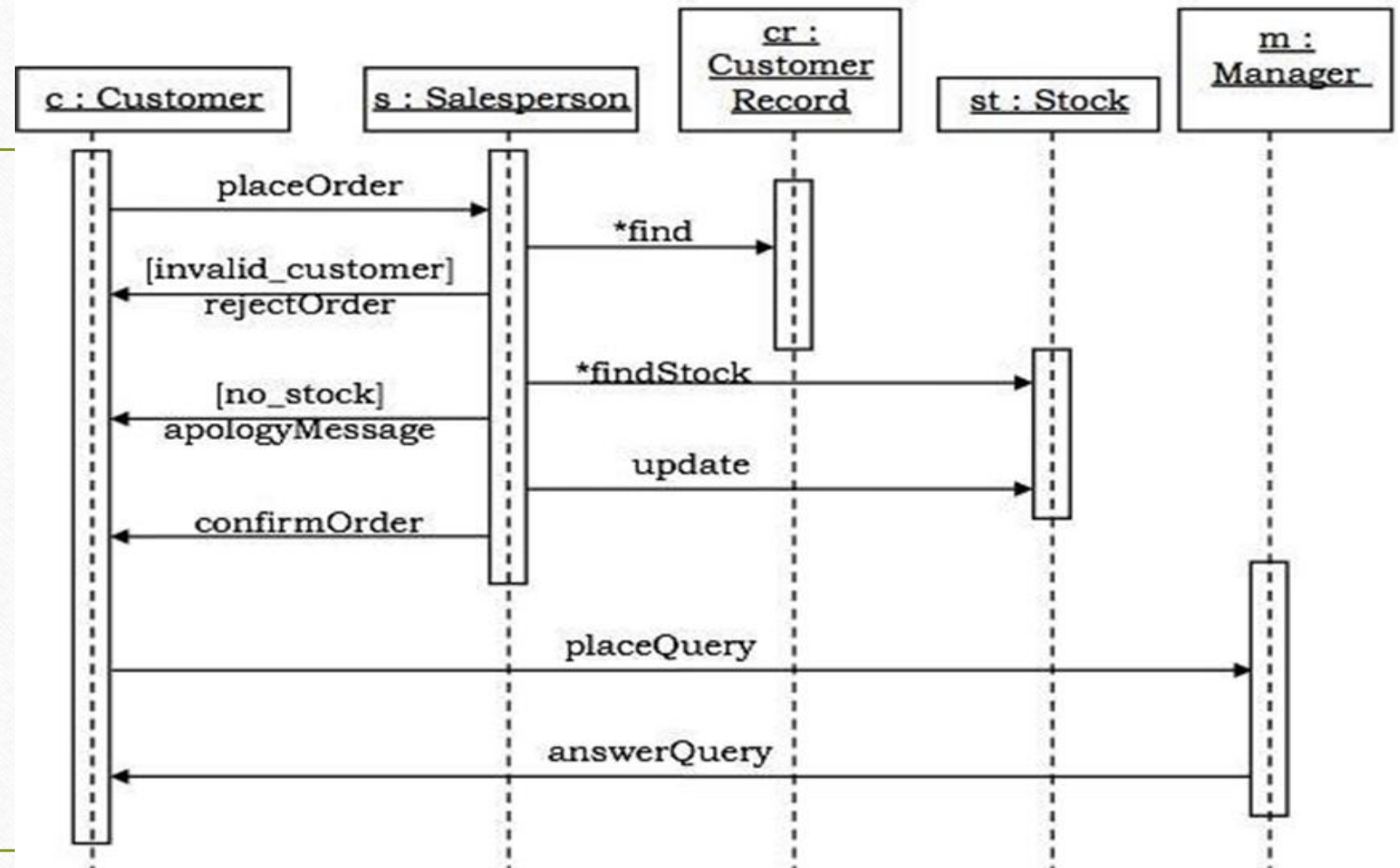
UML Behavioral Diagrams...

■ Sequence Diagrams

- ✓ Sequence diagrams are interaction diagrams that illustrate the ordering of messages according to time.
- **Notations:** These diagrams are in the form of **two-dimensional charts**.
- ✓ The objects that initiate the interaction are placed on the x-axis.
- ✓ The messages that these objects send and receive are placed along the y-axis, in the order of increasing time from top to bottom.

UML Behavioral Diagrams...

- ✓ **Example** : A sequence diagram for the **Automated Trading House System** is shown in the following figure.

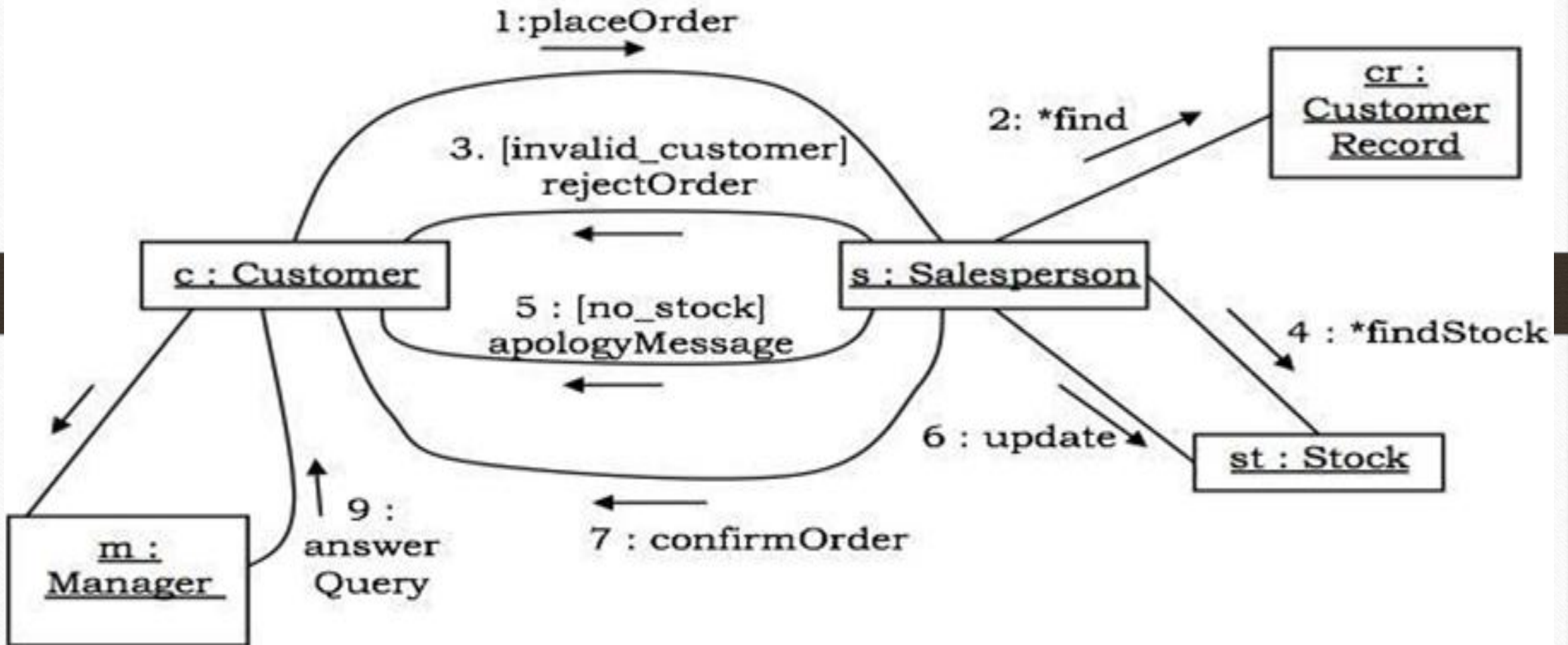


UML Behavioral Diagrams...

■ Collaboration Diagrams

- ✓ **Collaboration diagrams** are interaction diagrams that illustrate the structure of the **objects** that **send** and **receive** messages.
- **Notations** : In these diagrams, the objects that participate in the interaction are shown using vertices.
- The **links** that **connect** the objects are used to **send and receive messages**.
- *The message is shown as a labeled arrow.*
- ✓ Example : Collaboration diagram for the Automated Trading House System is illustrated in the figure below.

UML Behavioral Diagrams...



UML Behavioral Diagrams...

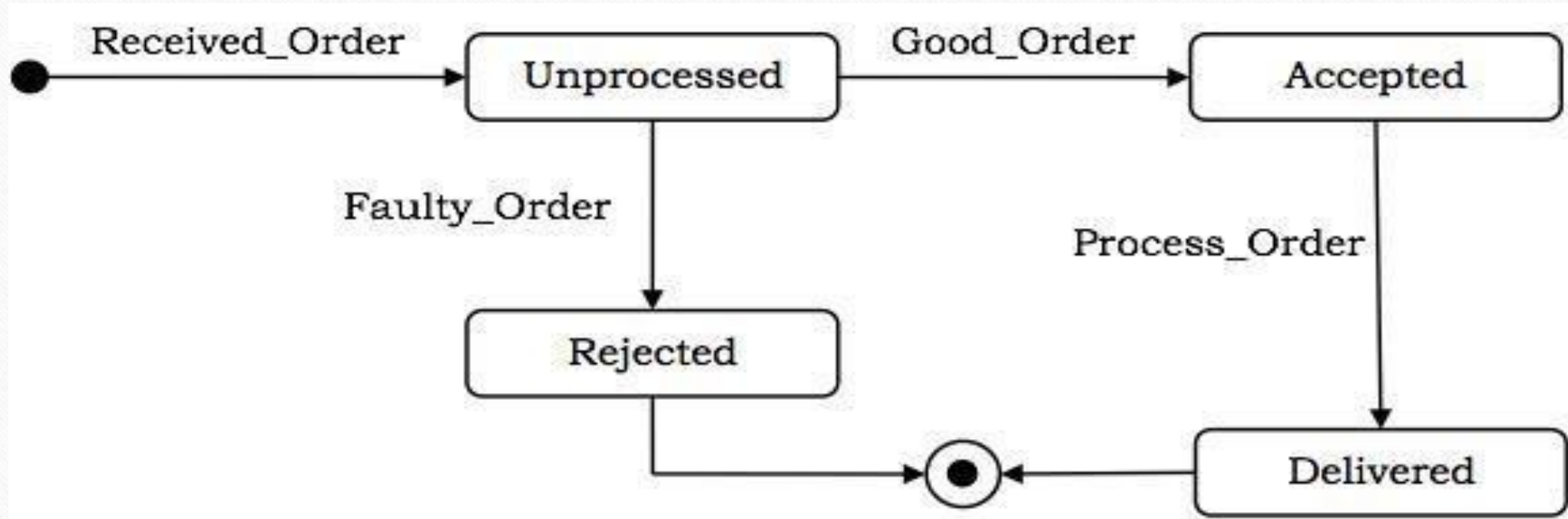
■ State–Chart Diagrams

- A **state–chart diagram** shows a state machine that **depicts** *the control flow of an object from one state to another.*
- A state machine portrays the sequences of states which an object undergoes due to **events** and **their responses to events.**
- *State–Chart Diagrams comprise of:*
 - **States: Simple or Composite**
 - **Transitions between states**
 - **Events causing transitions**
 - **Actions due to the events**

UML Behavioral Diagrams...

- **State-chart diagrams** are used for modeling objects which are reactive in nature.

Example: In the **Automated Trading House System**, let us model Order as an object and trace its sequence. *The following figure shows the corresponding state-chart diagram.*



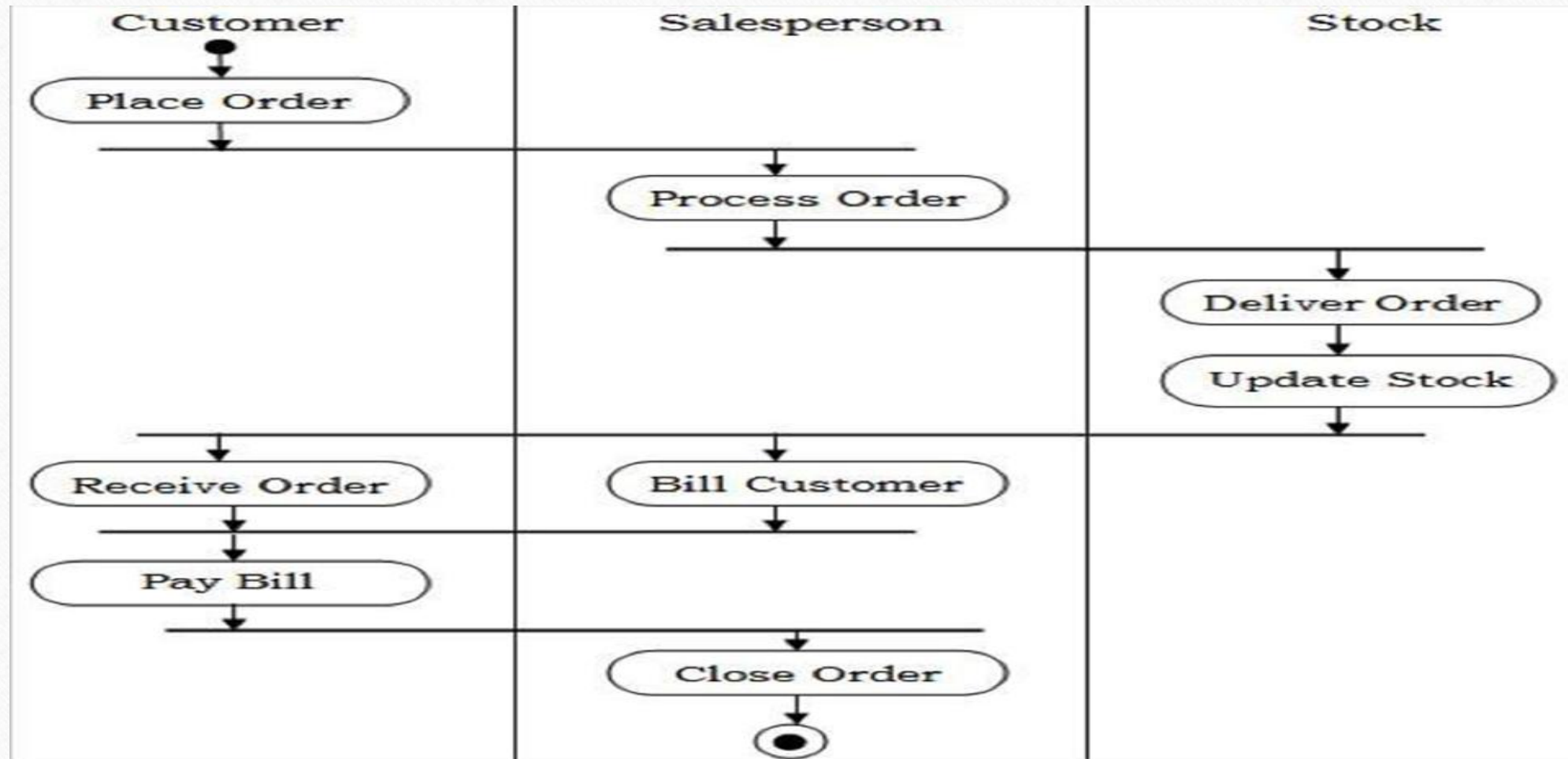
UML Behavioral Diagrams...

■ Activity Diagrams

- An **activity diagram** depicts the **flow of activities** which are **ongoing non-atomic** operations in a state machine.
- Activities result in actions which are atomic operations.
- **Activity diagrams comprise of:**
 - Activity states and action states
 - Transitions
 - Objects
- *Activity diagrams are used for modeling:*
 - Workflows as viewed by actors, interacting with the system.
 - details of operations or computations using flowcharts.

UML Behavioral Diagrams...

- **Example:** The following figure shows an activity diagram of a portion of the **Automated Trading House System**.



END OF CHAPTER - FIVE